Arx-Net: Platform Analysis & Comprehensive Documentation

Part 1: Platform Analysis & Market Context

What Problems It Solves

Understanding graph theory, tree data structures, and complex pathfinding algorithms is traditionally abstract and mathematically dense. Students, educators, and developers often struggle to visualize how algorithms like Dijkstra's, Bellman-Ford, or complex AVL tree rotations dynamically update node states and edge traverses. Arx-Net bridges the gap between theoretical algorithms and visual execution by providing a dynamic sandbox environment. It solves the problem of static, non-interactive learning tools by allowing users to instantly build, modify, and analyze graphs while watching algorithms execute step-by-step in real time, accompanied by live mathematical breakdowns.

Preexisting Solutions

Historically, users looking for graph visualization and algorithm testing have had to choose between three flawed categories of tools:

- Educational Visualizers (e.g., VisuAlgo): Web-based tools for visualizing algorithms. While helpful, they often restrict users to predefined graphs, offer clunky manual graph creation, and lack deep procedural generation.
- Enterprise Network Analyzers (e.g., Gephi, Cytoscape): Powerful, heavy-duty desktop applications built for data science. However, they have a steep learning curve, require installation, and entirely lack step-by-step algorithmic execution logs (they just output the final result).
- Programmatic Renderers (e.g., Graphviz): Excellent for rendering graphs via the DOT language, but they are completely static and lack real-time interactive node manipulation or algorithmic stepping.

Why Arx-Net is Better

Arx-Net seamlessly integrates the freeform canvas manipulation of a whiteboard with the rigorous programmatic execution of an IDE. Unlike standard visualizers, it allows for infinite procedural generation and robust custom text input, making graph creation instantaneous. Unlike enterprise software like Gephi, it is lightweight, browser-based, and focused explicitly on algorithm education and step-by-step logic breakdown. The ability to manipulate a node mid-execution or modify edge weights on the fly—and immediately see the topological and mathematical impact—makes it a vastly superior sandbox for dynamic learning and experimentation.

Competitive Analysis Table

Feature / Capability	**Arx-Net**	**VisuAlgo (Educational)**	**Gephi (Enterprise)**	**Graphviz (Programmatic)**
Real-Time Algorithmic Stepping	Yes (w/ logic breakdown)	Yes (limited scope)	No	No
Custom Graph Interaction (Drag/Drop)	Yes (Highly interactive)	Limited	Yes	No
Procedural Graph/Tree Generation	Yes (Extensive parameters)	No	Yes	No
Environment / Setup	Zero-install (Browser)	Zero-install	Heavy Desktop App	CLI / Desktop
Primary Focus Area	Interactive Algorithm Sandbox	Static Visualization	Big Data Analysis	Static Graph Rendering

Part 2: Comprehensive Documentation & Feature Guide

1. Introduction

Arx-Net is a dynamic, open-source graph and tree visualization tool built using HTML, CSS, and vanilla Javascript with D3.js. It is strictly designed to help users interactively build, modify, and analyze graphs and trees. The platform runs various path-finding, flow, and structural algorithms in real-time while providing step-by-step explanations, making it an excellent tool for learning and experimentation.

2. General Features & Interface

Header Controls (Main Toolbar)

- Guide / About (i icon): Opens a detailed guide modal directly inside the application explaining core interactions. This box also outlines the default tree name generation protocols.
- Sidebar Toggle: Shows or hides the left-hand graph generation and management panel.
- Generic Explanations: Resets the right-hand explanation panel to view generic, theoretical algorithm definitions instead of graph-specific execution logs.
- Display Modes: Toggle between Fullscreen mode and Light/Dark visual themes using the Sun/Moon icon.
- Import / Export (JSON): Use the Upload and Download buttons to save your current workspace state as JSON files or import existing ones. Note: Uploading appends to the current workspace and does not replace existing structures.

Outliner Controls (Left Panel)

The outliner provides a structured, hierarchical list of all generated graphs in the workspace.

- Global Controls:
    - Show/Hide All: Minimizes or maximizes all graphs simultaneously.
    - Delete All: Clears the entire workspace (requires confirmation).
- Individual Graph Management:
    - Rename: Double-click a graph name in the list to rename it. Names must be unique; the system will automatically append .001 if a duplicate name is provided.
    - Focus: Click the target icon next to a graph to immediately center the viewport on that specific graph window.
    - Context Menu (Right-Click): Right-click a graph's name to Duplicate or Delete it. When duplicating, you can alter the fundamental properties by changing the global Directed and Weighted checkboxes before executing the duplication.

Graph Viewport (Canvas Window)

Each generated graph lives inside an interactive, windowed container.

- Real-time Statistics Update: The canvas dynamically tracks and displays live topological metrics, including the exact Vertex and Edge counts, ensuring you always understand the scale of your current graph.
- Window Controls:
    - Resize: Click and drag the handles at the four corners of the container to resize the window.
    - Grid Toggle: A checkbox in the header allows you to show or hide the background grid lines.
    - Log Toggle: Click the book icon to hide/show the algorithm explanation panel linked to this specific graph.
    - Close Button: Click the 'X' to close/hide the graph window.
- Interactive Dragging: Use the Left Mouse Button (LMB) to click and drag vertices manually to arrange them.
- Camera Controls: Pan the canvas using the Middle Mouse Button, Ctrl + LMB, or by double-clicking and dragging an empty area. Zoom in and out smoothly using the scroll wheel.
- Visibility Optimizations: The engine provides specialized, highly-optimized visual rendering configurations for directed edges to prevent visual clutter when rendering dense topologies.

3. Graph Generation & Editing

Custom Graph Input

Arx-Net provides an intuitive text-based generation tool inside the left sidebar.

- Graph Name: Optional. Will default to a sequential generic name if left blank.
- Vertices Input: Explicitly define vertices (e.g., A, B, C). This is optional but useful for creating disconnected nodes.
- Edges Syntax: Edges can be specified in several versatile formats:
    - (A,B,5): Edge from vertex A to B with a weight of 5.
    - AB-4: Edge from vertex A to B with a weight of -4.
    - 12-6: Edge from vertex 1 to 2 with a weight of -6.
    - Native Python Adjacency/Edge list formatting (e.g., {"A": ["b", 2]}).
- Graph Properties Checkboxes: Before generation, toggle Directed and Weighted states.

Procedural Generation (Random Graph/Tree)

Click Customize parameters to open procedural generation settings:

- Generators: Choose between generating a generic "Graph" or a specific "Tree" topology.
- Node & Edge Count: Define the exact number of vertices and edges.
- Weight Bounds: Define the Min and Max values for edge weights.
- Graph Constraints: Toggle Connected Graph, Self Loops, and Duplicate Edges. The system calculates maximum/minimum theoretical edge counts dynamically based on these constraints to guide you.
- Quick Presets: Use Generate simple random graph or Generate complex random graph to automatically populate the parameters with optimized presets.

Real-Time Canvas Editing (Context Menus)

Once a graph is rendered, you can modify it on the fly using right-click context menus.

- Right-click the Background:
  - Create new vertex: Prompts for an ID and adds a new isolated vertex.
  - Rearrange nodes: Re-triggers the auto-layout function to organize nodes cleanly.
  - Save as PNG / Transparent PNG: Exports the specific graph view as an image.
  - Maximize / Center / Dock Left: Window snapping controls (desktop only).
- Right-click a Vertex:
  - Color this vertex: Opens a color picker to visually distinguish and highlight the selected node.
  - Change node value: Renames the vertex (updates all underlying edge references dynamically).
  - Create new edge from this vertex: Prompts for a target vertex ID and weight, drawing a new edge.
  - Delete this vertex: Removes the vertex and all connected edges.
  - Delete this vertex and its subtree: Recursively prunes the current vertex along with all of its downstream children from the graph.
- Right-click an Edge:
  - Change edge weight: Updates the numerical weight value.
  - Delete this edge: Removes the connection.

4. Supported Algorithms

Arx-Net features a comprehensive suite of graph algorithms. Selecting an algorithm from a graph's header dropdown executes it instantly. The visual state updates dynamically, and the right-hand panel populates with real-time, step-by-step logic and mathematical breakdowns.

- Breadth-First Search (BFS): Explores the graph layer by layer using a Queue.
- Depth-First Search (DFS): Explores as far as possible along each branch before backtracking using a Stack/Recursion.

- Dijkstra's Shortest Path: Finds the shortest path between a source node and all other nodes using a priority queue (requires non-negative edge weights).
- Floyd-Warshall Algorithm: Computes the shortest paths between all pairs of vertices using Dynamic Programming matrix calculations.
- Bellman-Ford Algorithm: Finds shortest paths from a single source and safely handles graphs with negative edge weights.
- Minimum Spanning Tree (MST): Highlights the spanning tree with the lowest total edge weight (utilizes Kruskal's or Prim's depending on the execution context).
- Topological Sorting: Linearly orders the vertices of a Directed Acyclic Graph (DAG) such that for every directed edge u v, u comes before v.
- Strongly Connected Components (SCC): Identifies maximal subgraphs where every vertex is reachable from every other vertex (utilizes Kosaraju's or Tarjan's algorithms).
- Maximum Flow: Computes the maximum possible flow from a source to a sink vertex (utilizes the Ford-Fulkerson or Edmonds-Karp methods).
- Biconnected Components: Identifies maximal biconnected subgraphs by tracking articulation points.

5. Supported Tree Types & Usage

Arx-Net includes a dedicated procedural generator for Tree data structures.

Because trees are mathematically a subset of graphs, all graph algorithms listed above can be executed on these trees. For example, Dijkstra's algorithm will successfully find the shortest path between any two nodes on a Weighted B-Tree, and BFS/DFS represent standard level-order and depth-first tree traversals.

The platform natively supports the generation of the following specific tree topologies:

Tree Type	Description
Regular Tree	A standard unconstrained tree where a parent can have any number of children.
Binary Tree	A tree where each node has at most two children.
Binary Search Tree (BST)	A binary tree where the left child is smaller and the right child is larger than the parent node.
AVL Tree	A strictly self-balancing binary search tree where the heights of the two child subtrees of any node differ by at most one.
2-3 Tree	A search tree where every node with children has either 2 or 3 children.
2-3-4 Tree	A self-balancing tree allowing nodes with up to 4 children.
B-Tree	A generalized, self-balancing search tree allowing nodes with more than two children, heavily used in databases.
B+ Tree	A variant of the B-Tree where all data

	pointers are stored exclusively at the leaf level, linked for efficient sequential access.

Tree Validators

When interacting with specific structural trees like BSTs and AVLs, you can verify their structural integrity at any time. Right-click the canvas background and select "Check BST" or "Check AVL" to run a programmatic validation that ensures the tree meets all necessary mathematical constraints, reporting the result in the console.

6. Tree Insertion Algorithms

While Arx-Net allows manual node and edge creation through the right-click interface, understanding the underlying algorithmic mechanics of tree insertion is critical when constructing or verifying specific tree types like Binary Trees, BSTs, AVL trees, and B-Trees.

Binary Tree Insertion

In a standard binary tree with no ordering properties, insertion is typically performed in level-order to maintain a complete tree structure.

1. Perform a Breadth-First Search (BFS) starting from the root using a queue.

2. Check the front node in the queue.

3. If it lacks a left child, insert the new node as the left child.

4. If it lacks a right child, insert the new node as the right child.

5. If it has both children, push them into the queue and repeat.

Binary Search Tree (BST) Insertion

Insertion in a BST must maintain the property: Left Child < Parent < Right Child.

1. Base Case: If the tree is empty, the new node becomes the root.

2. Traversal: Compare the new value to the current node (starting at the root).

3. If the new value is less than the current node, move to the left child.

4. If the new value is greater than the current node, move to the right child.

5. Insertion: Repeat steps 2-4 until you reach an empty spot (a null reference). Insert the new node as a leaf at that exact location.

(Note: Standard BSTs do not allow duplicate values, so if the new value equals an existing node, the insertion is rejected).

Balanced Tree Insertion (AVL Trees)

AVL Trees are strict, self-balancing Binary Search Trees. Every node maintains a Balance Factor, which is the difference in height between its left and right subtrees (Height(Left) - Height(Right)). The balance factor must always be -1, 0, or 1.

Insertion follows standard BST insertion logic, but is immediately followed by a balancing phase:

1. Insert the node as a leaf using standard BST logic.

2. Update Heights: Traverse back up the tree from the newly inserted leaf to the root, updating the height of every ancestor node.

3. Calculate Balance Factor: At each ancestor, calculate its Balance Factor.

4. Perform Rotations: If any ancestor's balance factor becomes greater than 1 or less than -1, the tree is unbalanced, and a rotation must be performed:

- Left-Left Case (LL): The new node was inserted into the left subtree of the left child. Fixed with a Right Rotation.
- Right-Right Case (RR): The new node was inserted into the right subtree of the right child. Fixed with a Left Rotation.
- Left-Right Case (LR): The new node was inserted into the right subtree of the left child. Fixed by performing a Left Rotation on the left child, followed by a Right Rotation on the unbalanced node.
- Right-Left Case (RL): The new node was inserted into the left subtree of the right child. Fixed by performing a Right Rotation on the right child, followed by a Left Rotation on the unbalanced node.

B-Tree Insertion

A B-Tree is a self-balancing search tree where nodes can have multiple keys and more than two children, keeping data sorted and allowing searches, sequential access, insertions, and deletions in logarithmic time.

1. Search: Traverse down the tree to find the correct leaf node for insertion.

2. Insert into Leaf: Insert the new key into the leaf node in sorted order.

3. Overflow Check: If the leaf node exceeds the maximum allowed keys (based on the tree's order), it splits into two nodes.

4. Promotion: The median key of the split node is promoted and inserted into its parent node.

5. Propagation: If the parent also overflows, the splitting and promotion propagate upwards. If the root overflows, it splits, and a new root is created, increasing the tree's overall height.

B+ Tree Insertion

A B+ Tree operates similarly to a B-Tree, but all data keys must reside at the leaf level, and leaves are linked together sequentially.

1. Search: Traverse down to find the correct leaf node.

2. Insert into Leaf: Insert the key into the leaf.

3. Overflow Check: If the leaf overflows, split it into two leaf nodes.

4. Copy Up: Unlike a B-Tree, the median key is copied up to the parent index node (rather than moved), ensuring it still exists at the leaf level for sequential traversal.

5. Internal Node Splitting: If internal nodes overflow, they split exactly like standard B-Tree nodes (moving the median key up).